

Hello. I want to build a Windows installer/updater with torrent download support.

Use **libtorrent-rasterbar** directly for torrent downloading.

Main goal:

Create an installer/downloader system that can:

1. Download required files through torrent.
2. Automatically load public tracker list from: `https://raw.githubusercontent.com/ngosang/trackerslist/master/trackers_all.txt`
3. Support DHT.
4. Ask user for download folder.
5. Ask user for unpack/install folder.
6. Verify downloaded files after download.
7. Unpack multi-volume 7-Zip archives into the selected unpack/install folder.
8. Avoid long path problems for Skyrim modlists.
9. Show progress and clear errors to the user.

Important Skyrim modlist path requirement:

Skyrim modlists can contain very deep folder paths. Because of this, the installer must ask the user to select a **short final unpack/install path**, for example:

```
```text id="w6z73e" D:\Sky D:\Modlist E:\SkyrimPack
```

Do not suggest long paths like:

```
```text id="k8tnvz"  
C:\Users\User\Downloads\SomeVeryLongInstallerFolder\Modlist\Output
```

The unpack/install folder should be a short folder close to the disk root.

The installer should ask the user where to unpack/install, but it should guide them to select only a short final folder on a chosen disk, for example `D:\Sky`.

Unpacking/extraction must not use a temporary folder on another disk. Extraction temp files, final output, and archive processing should stay on the same disk as the selected unpack/install folder when possible.

Recommended architecture:

- Main installer UI:
- Lets user choose download folder.
- Lets user choose unpack/install folder.

- Warns if unpack/install path is too long.
- Warns if unpack/install folder is inside deeply nested user folders.
- Starts the downloader/helper.
- Shows overall progress.
- Runs verification after download.
- Runs extraction after verification.
- Downloader/helper app:
 - Uses libtorrent-rasterbar.
 - Accepts `.torrent` file or magnet link.
 - Downloads files to the selected download folder.
 - Loads additional trackers from the online tracker list.
 - Supports DHT.
 - Reports progress, speed, peers, ETA, and errors.
- Tracker manager:
 - Downloads tracker list from: `https://raw.githubusercontent.com/ngosang/trackerslist/master/trackers_all.txt`
 - Parses tracker URLs.
 - Ignores empty lines.
 - Ignores invalid tracker URLs.
 - Adds valid trackers to the torrent session or torrent handle.
 - Continues without failing if tracker list cannot be downloaded.
 - Logs tracker list success or failure.
- Verifier:
 - Checks torrent completion.
 - Performs additional SHA256 verification from a manifest file.
- Extractor:
 - Uses bundled 7-Zip command-line executable, for example `7z.exe`, `7za.exe`, or `7zz.exe`.
 - Extracts from the first archive part, for example `archive.7z.001`.
 - Extracts into the selected short unpack/install folder.
 - Ensures extraction does not use a temporary folder on another disk.

Expected installer flow:

1. Installer starts.

2. User chooses download folder.
 3. User chooses unpack/install folder.
 4. Installer validates unpack/install folder:
 5. Path should be short.
 6. Path should be close to disk root.
 7. Example valid path: `D:\Sky`
 8. Warn if path is too long.
 9. Warn if path is under `C:\Users\...`
 10. Warn if target disk has insufficient free space.
 11. Installer downloads or loads manifest file.
 12. Manifest contains:
 13. torrent path or magnet link
 14. expected file list
 15. file sizes
 16. SHA256 hashes
 17. archive first part name, for example `modpack.7z.001`
 18. extraction settings
 19. Installer downloads tracker list from: `https://raw.githubusercontent.com/ngosang/trackerslist/master/trackers_all.txt`
 20. Downloader starts torrent session using libtorrent.
 21. Downloader enables DHT.
 22. Downloader adds trackers from tracker list.
 23. Downloader saves all torrent files to the selected download folder.
 24. Downloader shows:
 25. total progress
 26. download speed
 27. upload speed
 28. peer count
 29. current state
 30. ETA if possible
 31. DHT status if possible
 32. tracker count if possible
 33. After torrent reaches 100%, run torrent recheck if supported cleanly.
 34. Then verify downloaded files using SHA256 from manifest.
 35. If verification passes, start 7-Zip extraction.
 36. Extract multi-volume archive by running extraction on the first part:
- ```
bat id="pyqm6a" 7z.exe x "D:\Downloads\modpack.7z.001" -o"D:\Sky" -y
```

17. Ensure extraction target is the selected unpack/install folder.
18. Do not extract to a temporary folder on another disk.
19. Show extraction progress if possible.
20. After extraction, optionally clean temporary downloaded archives.
21. Show success message.

Folder selection requirements:

Download folder:

- Ask user to select download folder.
- This is where torrent files and archive parts are downloaded.
- Support resume if files already exist.
- Check write permission.
- Check free disk space.
- Warn if folder is too long.

Unpack/install folder:

- Ask user to select final unpack/install folder.
- This folder should be short, for example `D:\Sky`.
- This folder should not be deeply nested.
- Strongly warn against long paths.
- Check write permission.
- Check free disk space.
- Extraction must write directly to this folder.
- Extraction must not use another disk for temporary extraction work.
- If temporary files are needed, place them inside the selected unpack/install folder or another short folder on the same drive.

Same-disk extraction requirement:

The extractor must avoid cross-disk unpacking behavior.

Rules:

- Final unpack/install folder is the source of truth.
- Temporary extraction folder, if needed, must be on the same drive as final unpack/install folder.
- Do not use `%TEMP%` if `%TEMP%` is on another drive.
- Do not unpack to `C:\Users\...\AppData\Local\Temp` and then move to `D:\Sky`.
- Avoid copy/move between disks after extraction.
- Prefer direct extraction using 7-Zip `-o"D:\Sky"`.
- If staging is required, use something like:

```
```text id="lmljpk"
D:\Sky\.install_temp
```

Path length requirements:

- Detect long target paths before extraction.
- Warn user if selected folder path is too long.
- Recommend short root-level folder such as `D:\Sky`.
- Normalize paths.
- Avoid adding unnecessary nested folders.
- Ensure manifest paths are safe.
- Prevent path traversal using entries like `..\`.
- Do not allow extraction outside selected unpack/install folder.
- Optionally enable Windows long path support detection, but do not rely on it.

Torrent requirements:

- Use libtorrent-rasterbar.
- Support magnet links.
- Support `.torrent` files.
- Enable DHT.
- Enable peer exchange if available.
- Enable local service discovery if available.
- Add trackers from the online tracker list.
- Continue if tracker list download fails.
- Resume partial downloads.
- Recheck existing files before redownloading.
- Save resume data if possible.
- User must be able to cancel download.
- On cancel, stop torrent session safely.
- Show readable torrent errors.

Tracker list requirements:

- Download tracker list from: `https://raw.githubusercontent.com/ngosang/trackerslist/master/trackers_all.txt`
- Parse line by line.
- Trim whitespace.
- Ignore empty lines.
- Ignore invalid URLs.
- Accept tracker protocols such as:
 - `udp://`
 - `http://`
 - `https://`
 - `wss://` if supported
- Remove duplicates.
- Add trackers to torrent.
- Log number of trackers loaded.

- Do not fail the whole installation if tracker list is unavailable.

Verification requirements:

- Torrent completion alone is not enough.
- After torrent finishes, verify downloaded files against manifest SHA256 hashes.
- If a file hash does not match:
 - mark verification as failed
 - show which file failed
 - offer retry/recheck if possible
- Do not extract if verification fails.

7-Zip extraction requirements:

- Support multi-volume archives:
 - .7z.001
 - .7z.002
 - .7z.003
 - etc.
- Start extraction from .7z.001.
- All volume parts should be in the same folder.
- Use full-path extraction mode.
- Extract into the selected unpack/install folder.
- Capture stdout/stderr from 7-Zip.
- Detect extraction failure by exit code.
- Show clear error if:
 - 7-Zip executable is missing
 - archive part is missing
 - archive volume is missing
 - archive is corrupted
 - disk space is insufficient
 - access denied
 - target folder is not writable
 - target path is too long

Suggested manifest format:

```
```json id="fcupvw" { "version": "1.0.0", "torrent": { "type": "magnet", "source": "magnet:?xt=urn:btih:..." },
"trackers": { "use_online_tracker_list": true, "tracker_list_url": "https://raw.githubusercontent.com/ngosang/
trackerslist/master/trackers_all.txt", "continue_if_tracker_list_fails": true }, "torrent_features": { "enable_dht":
true, "enable_pex": true, "enable_lsd": true }, "download": { "ask_user": true }, "install": { "ask_user": true,
"prefer_short_root_path": true, "example_path": "D:\\Sky", "max_recommended_path_length": 20,
"same_disk_temp_only": true }, "files": [{ "path": "modpack.7z.001", "size": 2147483648, "sha256": "..."},
{ "path": "modpack.7z.002", "size": 2147483648, "sha256": "..."}], "extract": { "first_archive_part": "modpack.
7z.001", "target_subfolder": "", "cleanup_after_success": false, "use_same_disk_temp": true } }
```

Implementation plan:

Phase 1 – Project structure

Create a clean project structure:

```
```text id="frrv81"
```

```
InstallerApp/  
  src/  
    ui/  
    downloader/  
    tracker/  
    verifier/  
    extractor/  
    manifest/  
    paths/  
    logging/  
  third_party/  
    libtorrent/  
    7zip/  
  tests/  
  docs/
```

Phase 2 — Manifest loader

Implement manifest loading and validation.

The manifest loader should:

- Read JSON manifest.
- Validate required fields.
- Validate torrent source.
- Validate tracker settings.
- Validate torrent feature settings.
- Validate file list.
- Validate extraction settings.
- Validate install path rules.
- Return clear errors for invalid manifests.

Phase 3 — Folder selection and path validation

Implement folder selection before download starts.

The UI should ask for:

1. Download folder.

2. Unpack/install folder.

Path validator should:

- Check folder exists or can be created.
- Check write permission.
- Check available free space.
- Detect drive letter.
- Check if unpack/install path is short.
- Warn if unpack/install path is deeply nested.
- Warn if unpack/install path is probably unsafe for Skyrim modlists.
- Recommend examples like `D:\Sky`.
- Ensure extraction temp folder is on the same drive as unpack/install folder.
- Prevent path traversal.

Create a clean interface like:

```
```cpp id="wjcsyx" struct PathValidationResult { bool ok; bool warning; std::string message;
std::filesystem::path normalizedPath; std::string driveRoot; };
```

```
class IPathValidator { public: virtual PathValidationResult ValidateDownloadFolder(const
std::filesystem::path& path) = 0; virtual PathValidationResult ValidateInstallFolder(const
std::filesystem::path& path) = 0; virtual bool IsSameDrive(const std::filesystem::path& a, const
std::filesystem::path& b) = 0; };
```

### Phase 4 – Tracker manager

Implement tracker list loading.

Tracker manager should:

- Download tracker list URL.
- Parse line by line.
- Validate protocols.
- Remove duplicates.
- Return tracker list.
- Log count of trackers.
- Continue safely if URL is unavailable.

Create a clean interface like:

```
```cpp id="zlpnbi"
class ITrackerProvider {
public:
    virtual std::vector<std::string> LoadTrackers(const std::string& url) = 0;
};
```


Phase 5 — Torrent downloader

Implement torrent downloading using libtorrent-rasterbar.

Downloader should support:

- Add torrent from magnet link.
- Add torrent from `.torrent` file.
- Set selected download folder.
- Enable DHT.
- Enable peer exchange if available.
- Enable local service discovery if available.
- Add trackers from tracker manager.
- Resume partial downloads.
- Report progress.
- Report speed.
- Report peer count.
- Report torrent state.
- Report DHT status if possible.
- Report tracker count if possible.
- Cancel safely.
- Pause/resume if possible.
- Save resume data if possible.

Create a clean interface like:

```
```cpp id="ie9h9g" class ITorrentDownloader { public: virtual void Start(const DownloadConfig& config) = 0;
virtual void Cancel() = 0; virtual void Pause() = 0; virtual void Resume() = 0; virtual DownloadStatus
GetStatus() const = 0; };
```

## Phase 6 – Verification

Implement SHA256 verification.

Verifier should:

- Iterate over manifest files.
- Check that every file exists in selected download folder.
- Check file size.
- Compute SHA256.
- Compare with manifest hash.
- Return detailed result per file.
- Stop extraction if any file fails.

## Phase 7 – 7-Zip extractor

Implement extraction by launching bundled 7-Zip executable.

Extractor should:

- Locate `7z.exe`, `7za.exe`, or `7zz.exe`.
- Build safe command-line arguments.
- Use selected download folder as archive source.
- Use selected unpack/install folder as target.
- Prefer direct extraction:

```
```bat id="odcf8k"  
7z.exe x "D:\Downloads\modpack.7z.001" -o"D:\Sky" -y
```

- If temp folder is needed, use same-disk temp folder:

```
text id="xzt6pj"  
D:\Sky\.install_temp
```

- Never use temp folder on another disk.
- Capture output.
- Parse progress if possible.
- Return success/failure based on exit code.
- Log stdout/stderr.

Phase 8 — Installer UI

Implement user interface.

UI should show:

- Download folder selector.
- Unpack/install folder selector.
- Warning text recommending short path like `D:\Sky`.
- Current stage:
- Loading manifest
- Loading trackers
- Downloading
- Checking files
- Extracting
- Completed
- Progress bar.
- Current download speed.
- Upload speed.
- Peers.
- ETA.
- DHT status.
- Tracker count.
- Current file/stage text.

- Cancel button.
- Retry button on failure.
- Open log button.

Phase 9 — Logging

Add detailed logs.

Log should include:

- App version.
- Manifest path/source.
- Download folder.
- Unpack/install folder.
- Drive letters used.
- Same-disk temp validation result.
- Tracker list URL.
- Number of trackers loaded.
- DHT enabled/disabled status.
- Torrent state changes.
- Torrent engine errors.
- Verification result.
- 7-Zip command used, without exposing sensitive data.
- 7-Zip exit code.
- Extraction result.

Phase 10 — Error handling

Handle these errors cleanly:

- No internet connection.
- Tracker list unavailable.
- Invalid tracker URL.
- Invalid magnet link.
- Invalid torrent file.
- DHT unavailable.
- No peers.
- Download stalled.
- Disk full.
- Permission denied.
- User cancelled.
- Hash mismatch.
- Missing archive volume.
- 7-Zip extraction failed.
- Target folder not writable.
- Target path too long.
- Extraction temp folder would be on another disk.

Phase 11 — Testing

Add tests or manual test scripts for:

- Manifest validation.
- Tracker list parsing.
- Tracker list unavailable.
- DHT enabled.
- Download folder validation.
- Unpack/install folder validation.
- Same-disk temp validation.
- Long path warning.
- Hash verification.
- Missing file detection.
- Wrong hash detection.
- Extraction success.
- Extraction failure.
- Cancel download.
- Resume download.
- Existing files recheck.
- Invalid target folder.

Phase 12 — Packaging

Package installer with:

- Main installer executable.
- Downloader/helper executable if separate.
- Required torrent runtime files.
- 7-Zip executable.
- License files for third-party components.
- Example manifest.
- Example config.

Important technical notes:

- Keep torrent logic separated from UI.
- Keep tracker loading separated from torrent logic.
- Keep path validation separated from extraction logic.
- Do not block UI thread.
- Prefer a helper process if the installer framework is not good for long-running torrent operations.
- Always verify SHA256 before extraction.
- Never extract unverified files.
- Make paths safe against path traversal.
- Normalize paths from manifest.
- Do not allow archive extraction outside selected unpack/install folder.
- Check available disk space before download and before extraction.
- Avoid long install paths because Skyrim modlists can contain deeply nested files.

- Include third-party licenses.

Deliverables:

1. Recommended architecture.
2. Source code skeleton.
3. Manifest parser.
4. Folder selection and path validation system.
5. Tracker list downloader/parser.
6. Libtorrent downloader implementation.
7. DHT support.
8. SHA256 verifier.
9. 7-Zip extractor wrapper.
10. Same-disk extraction/temp handling.
11. UI progress integration.
12. Logging system.
13. Error handling.
14. Build/package instructions for Windows.

Please first propose the final architecture and project layout, then implement the code step by step.